

Section 6 — In & Out: Serialization

Concept Notes

Pydantic handles data in two directions: converting external data into a validated model and converting a model back into plain data for storage, APIs, or files.

These two processes are commonly called **deserialization** and **serialization**.

Deserialization

Deserialization means converting raw external data into a validated Pydantic model.

The incoming data may come from:

- Python dictionaries
- JSON strings
- API requests
- Files
- External services

Pydantic checks and converts the incoming data before creating the model object.

The main methods are:

- `model_validate()` — validate Python data such as a dictionary
- `model_validate_json()` — validate data directly from a JSON string

Serialization

Serialization means converting a validated Pydantic model into a format that can be stored or sent somewhere else.

The main methods are:

- `model_dump()` — convert a model into a Python dictionary
- `model_dump_json()` — convert a model into a JSON string

`model_dump()`

`model_dump()` converts a Pydantic model into a Python dictionary.

It is useful when data needs to be:

- Stored in a database
- Modified as dictionary data
- Passed to another Python function
- Processed by another library

`model_dump_json()`

`model_dump_json()` converts a Pydantic model directly into a JSON string.

It is useful for:

- API responses
- Saving JSON data
- Sending data between systems
- Network communication

Include and Exclude Fields

During serialization, specific fields can be included or excluded.

This is useful when:

- Sensitive data should not be returned
- Only selected fields are required
- Internal fields should remain hidden

For example, a password can be excluded before sending user data in an API response.

Important Points

- Deserialization converts raw data into a validated Pydantic model.
- Serialization converts a model into plain Python data or JSON.

- `model_validate()` validates Python data such as dictionaries.
- `model_validate_json()` validates JSON strings.
- `model_dump()` returns a Python dictionary.
- `model_dump_json()` returns a JSON string.
- Fields can be selectively included or excluded during serialization.
- Serialization is commonly used in APIs, databases, files, and data exchange.
- Pydantic V2 uses `model_dump()` instead of the older `.dict()` method.
- Pydantic V2 uses `model_dump_json()` instead of the older `.json()` method.

Syntax

Validate Python Data

```
Model.model_validate(data)
```

Validate JSON Data

```
Model.model_validate_json(json_data)
```

Convert Model to Dictionary

```
model_object.model_dump()
```

Convert Model to JSON

```
model_object.model_dump_json()
```

Exclude Fields

```
model_object.model_dump(
    exclude = {fields}
)
```

Include Selected Fields

```
model_object.model_dump(
    include = {fields}
)
```